

Background

While there are numerous comorbidity indices and scales that have been developed in the past, nearly all of them are based on diseases that are typically associated with aging. Furthermore, the most popular indices of the Charlson comorbidity index and Elixhauser index are based on mortality, which is not very common and a good measure for younger populations. As such, these indices may not accurately and adequately capture the morbidity burden of young adults between the ages of 18 and 39.

Given this, there is a gap in appropriate comorbidity measures for young adults. To develop this possible young adult morbidity index (YAMI), the All of Us data may be utilized as it enables the use of a large population-based cohort of adults in the United States.

All of Us data is unique in the sense that it contains a comprehensive set of measures for health conditions for a nationally representative sample of young adults. Furthermore, the data includes health status measures, which is often not included in administrative datasets such as claims or electronic health record (EHR) data. The All of Us data is also linked with EHR data, which allows the possible use of ICD-10 and/or SNOMED codes in the future to develop further versions of the index. The index will be developed using a set of self-reported health conditions as predictors and self-reported health statuses as outcomes.

Methods

Data Selection

The All of Us web platform allows a relatively straightforward cohort builder tool, enabling a simplified way of querying data. With it, a couple of parameters are able to be input to subset the data appropriately. First, the age at consent variable is filtered to those between the ages of 18 and 39. Following that, a few concept sets, All of Us' way of having reproducible queries, are used to further subset the data. The first concept set contains demographic information, with those variables including gender, race, and ethnicity. The next concept set utilized overall health, which contained information such as general mental health and general health quality. The last concept set used was medical history, which contained data on many health statuses like the presence/current status of an enlarged prostate, osteoarthritis, or yeast infection.

Data Cleaning

Given that the data provided by *All of Us* was used, there was not a lot of data preparation and cleaning that needed to be done besides the typical variable type changes and value

conversion/mapping. A key note is that for questions regarding a person's current status of a comorbidity, missing values were changed to 'No' because of the question asking progression (ex. Age someone started smoking would be missing because they would not have been asked if they already answered that they never smoked). Additionally, an age variable had to be calculated by finding the difference between the date each person took the survey and their birth date.

The final dataset consisted of two types of *All of Us* data, with those being demographic and survey data. The survey data had to be formatted correctly to be able to get the survey question and response for each individual. To do this, the *person_id* and *answer* variables were used. These were the only variables needed because the *answer* values were formatted as 'Question: Response'. As such, the values were able to be parsed and labeled appropriately into *question* and *response* variables.

Because the data came with each question and response per person being its own row/observation, the data had to be transposed from long to wide format. This would make it easier to look at the presence of morbidities per person and easier to process the data. Duplicate rows also had to be removed to ensure that there were no issues with the transposing. With the parsed survey dataset containing the variables *person_id*, *question*, and *response*, the data was converted so that each person only had one row within the final dataset and there were binary variables for each comorbidity status.

The resulting dataset was then merged with some demographic variables by *person_id*. Additionally, a correlation check on variables was done to see if any variables could be removed. To do this, categorical variables were converted to binary dummy variables and the health outcome variable values ('Poor', 'Fair', 'Good', 'Very Good', 'Excellent') were numerically coded. These values were used based on **PRIOR PAPER NAME** to allow for correlations to be calculated. A cutoff value of 0.75 was used for this, and no comorbidities were able to be removed.

Analysis

The main aspect of the analysis used a final overall model that consisted of three steps. An important note is that binary outcomes were used throughout as C scores were used to compare model performance. Outcome values of 'Poor' and 'Fair' were labeled as 0 while values of 'Good', 'Very Good', and 'Excellent' were labeled as 1. Linear regression was trialed as well, but it was determined that utilizing C scores instead of R^2 values would be a simpler and more effective way to interpret models.

The first step used a cross-validated lasso model to identify relevant variables to move forward with. The *glmnet()* function from the *glmnet* package was used and it was found that using either the minimum or 1 standard error lambda value had minimal impact on relevant variables. This step returned a dataset specifying each variable in the dataset and its coefficient value.

Ordinal lasso regression, both linear and logistic, was also tried out in an attempt to utilize the full range of health status answers and preserve order and meaning between them. However, because of the large amount of input variables, the model took a very long time to run with each pass. Additionally, the coefficients and model result interpretations were much more difficult than simply using the resulting coefficients from regression models.

For step two, the variables with non-zero coefficients were passed into a logistic regression model using the *glm()* function from the *stats* package. The model result was then extracted and returned a dataset containing each of the non-zero variables previously identified, along with their corresponding coefficient and p-values.

Step three took this coefficient data and subsetting the variables based on their p-values. Since most of the variables had p-values smaller than 0.05, an alpha value of 0.01 was used. With this set of statistically significant variables, a final logistic regression model was created. From this model, the variables and their coefficient values were returned.

These coefficient values were then converted to integers to create an easily implementable scoring system. First, each coefficient was divided by the smallest absolute value coefficient. After this, each result was then rounded to the nearest integer value and then multiplied by negative one to make it so scores were mostly positive. This scoring system enabled the comparison of the proposed YAMI index versus the Charlson and Elixhauser indices by allowing the calculation of index scores for each of the indices for each person.

Once these scores were calculated for each person, logistic regression models were used to calculate C scores for each of the indices. K-fold cross validation was also used, where the data was randomly split into a number of subgroups defined by the user. For each fold, three models were created by using each index score to predict the outcome. The C scores for each of these models were then printed and stored, with a final overall average C score returned at the end for each index.

Additionally, there was some extra functionality created to look at various things regarding the relationship between the scoring of the different overall health outcome variables. The first looked at the presence of each comorbidity in the final model for each overall health variable, which allows the comparison of their importance across the board. The other aspect evaluated whether or not an overall health outcome variable could predict the others. To do so, YAMI scores were calculated for each variable and then linear regression models were used where one variable's score predicted another's.

Results

One of the important data cleaning aspects was splitting the question variable values into the appropriate question and answer variables.

	person_id	question_concept_id	question	answer_concept_id	answer	date
0	1937810	1585723	Overall Health: General Physical Health	1585724	General Physical Health: Excellent	2018-09-27
1	2743933	1585711	Overall Health: General Health	1585712	General Health: Excellent	2018-11-26
2	3070644	1585711	Overall Health: General Health	1585712	General Health: Excellent	2018-10-04
3	3200088	1585723	Overall Health: General Physical Health	1585724	General Physical Health: Excellent	2021-12-20
4	1760833	1585729	Overall Health: General Mental Health	903626	General Mental Health: Excellent	2020-03-06

Figure 1: Original Data Set

	person_id	question	response
0	1937810	General Physical Health	Excellent
1	2743933	General Health	Excellent
2	3070644	General Health	Excellent
3	3200088	General Physical Health	Excellent
4	1760833	General Mental Health	Excellent

Figure 2: Cleaned Question Values

The next important aspect of the cleaning was to transpose the data from long to wide.

	person_id	ADHD	Acid Reflux	Acne	Alcohol Or Drug	Allergies	Anemia	Anxiety	Asthma	Astigmatism	...	Skin Cancer	Skin Condition	Sleep Apnea	Spine Muscle Bone	Tinnitus	Type 2 Diabetes
0	1000093	NaN	NaN	Yes	NaN	No	NaN	Yes	NaN	NaN	...	NaN	No	NaN	NaN	NaN	NaN
1	1000095	NaN	Yes	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Yes	NaN	Yes	No	NaN
2	1000107	NaN	NaN	No	NaN	No	NaN	Yes	NaN	Yes	...	NaN	NaN	NaN	NaN	NaN	NaN
3	1000112	NaN	NaN	No	NaN	NaN	NaN	Yes	No	NaN	...	NaN	NaN	NaN	NaN	No	NaN
4	1000184	NaN	NaN	No	NaN	No	NaN	NaN	NaN	No	...	NaN	NaN	NaN	NaN	NaN	NaN

Figure 3: Data in Wide Format (before turning missing values to 'No')

Another thing that was checked was the correlation among variables, where it was found that no variable pairing exceeded the benchmark of 0.75.

	age_years	ADHD	Acid Reflux	Acne	Alcohol Or Drug	Allergies	Anemia	Anxiety	Asthma	Astigmatism	...	Female	Male	Other	Asian	Black	Middle Eastern	More than one	White
0	19	0	0	0	0	0	0	1	0	1	...	0	0	0	0	0	0	0	0
1	23	1	0	0	1	0	0	1	0	0	...	0	0	0	0	0	0	0	0
2	35	0	0	0	0	1	0	0	0	0	...	0	0	0	0	0	0	0	0
3	29	0	0	0	0	1	0	0	0	0	...	0	0	0	0	0	0	0	0
4	25	0	0	0	0	0	0	0	0	1	...	0	0	1	0	0	0	0	0

5 rows × 96 columns

Figure 4: Data Prepared for Correlations

As stated before, ordinal lasso regression was tried out, but the idea was abandoned primarily after it was found that the runtime was much longer than expected.

	logit(P[Y<=1])	logit(P[Y<=2])	logit(P[Y<=3])	logit(P[Y<=4])
(Intercept)	-3.538324	-1.88980834	-2.557184e-01	1.79392816
age_years	0.000000	0.00000000	1.269153e-03	0.00000000
ADHD	0.000000	0.00000000	1.660153e-02	0.00000000
Acid Reflux	0.000000	0.28711232	3.365097e-01	0.15640401
Acne	0.000000	0.00000000	-1.150856e-04	0.00000000
Alcohol Or Drug	0.000000	0.00000000	0.000000e+00	0.00000000
Allergies	0.000000	0.00000000	0.000000e+00	0.00000000
Anemia	0.000000	0.12585534	1.355560e-01	0.00000000
Anxiety	0.000000	0.00000000	9.764195e-02	0.17059940
Asthma	0.000000	0.11403119	9.342296e-02	0.00000000
Astigmatism	0.000000	0.00000000	0.000000e+00	0.00000000
Atrial Fibrillation	0.000000	0.00000000	0.000000e+00	0.00000000
Bipolar	0.000000	0.06142027	6.203093e-02	0.00000000

Figure 5: Ordinal Lasso Regression Model Trial Results

The analysis section within *Methods* outlined the overall model structure, which consists of a lasso model and two logistic regression models.

```

# function to create lasso model based on desired outcome variable type
get_lasso_coef_overall <- function(dataframe, is_binary, is_lambda_min, max_df) {
  # make x and y datasets
  x_data <- dataframe |> select(-outcome)
  x_data <- data.matrix(x_data)
  y_data <- dataframe$outcome
  print(table(y_data))

  if (is_binary == 1) {
    # make outcome variable binary
    y_data <- replace(y_data, y_data == 1, 0)
    y_data <- replace(y_data, y_data == 2.05, 0)
    y_data <- replace(y_data, y_data == 3.58, 1)
    y_data <- replace(y_data, y_data == 4.7, 1)
    y_data <- replace(y_data, y_data == 5, 1)
    print(table(y_data))

    # lasso with cross validation - calculate lambda
    set.seed(123)
    model <- cv.glmnet(x_data, as.factor(y_data), alpha = 1, nfolds = 10, family = 'binomial',
                      dfmax = max_df)
    if (is_lambda_min == 1) {
      lambda_val <- model$lambda.min
      print(sprintf('Minimum Lambda -> %f', lambda_val))
    }
    else {
      lambda_val <- model$lambda.1se
      print(sprintf('1 SE Lambda -> %f', lambda_val))
    }
    #plot(model)

    # get coefficients - model with minimum lambda
    model_small_lambda <- glmnet(x_data, as.factor(y_data), lambda = lambda_val, alpha = 1, family = 'binomial',
                                dfmax = max_df)
    coef_result <- coef(model_small_lambda)
  }
  else {
    # lasso with cross validation - calculate lambda
    set.seed(123)
    model <- cv.glmnet(x_data, y_data, alpha = 1, nfolds = 10,
                      dfmax = max_df)
    if (is_lambda_min == 1) {
      lambda_val <- model$lambda.min
      print(sprintf('Minimum Lambda -> %f', lambda_val))
    }
    else {
      lambda_val <- model$lambda.1se
      print(sprintf('1 SE Lambda -> %f', lambda_val))
    }
    #plot(model)

    # get coefficients - model with minimum lambda
    model_small_lambda <- glmnet(x_data, y_data, lambda = lambda_val, alpha = 1,
                                dfmax = max_df)
    coef_result <- coef(model_small_lambda)
  }
  # return coefficients as dataframe
  coef_matrix <- as.matrix(coef_result)
  coef_matrix_df <- as.data.frame(coef_matrix)
  coef_matrix_df <- cbind(comorbidity = rownames(coef_matrix_df), coef_matrix_df)
  rownames(coef_matrix_df) <- 1:nrow(coef_matrix_df)
  colnames(coef_matrix_df) <- c('comorbidity', 'value')
  return(coef_matrix_df)
}

```

Figure 6: Lasso Model Function Code

```

# function to create regression models given numeric data and the coefficient values
create_first_model_overall <- function(dataframe, lasso_coefficients_df, is_logistic_model) {
  # check if age is included in final model
  is_age_included = FALSE
  if (lasso_coefficients_df[2,]$value != 0) {
    is_age_included = TRUE
  }

  # subset data to variables with a non-zero coefficient
  non_zero <- lasso_coefficients_df[lasso_coefficients_df['value'] != 0,][-1,]
  non_zero_comors <- non_zero$comorbidity
  num_non_zeros <- length(non_zero_comors)
  print(sprintf('Number of Non-Zero Comorbidities from Lasso Model -> %i', num_non_zeros))

  #-> since removing age from data

  # check presence of age for variable types
  if (is_age_included == FALSE) {
    # data <- dataframe |>
    #   select(c(all_of(non_zero_comors), outcome)) |>
    #   mutate(across(-outcome, as.factor))
  }
  #else {
    # data <- dataframe |>
    #   select(c(all_of(non_zero_comors), outcome)) |>
    #   mutate(across(-c(outcome, age_years), as.factor))
  }

  data <- dataframe |> select(c(all_of(non_zero_comors), outcome)) |>
    mutate(across(-outcome, as.factor))

  # drop unknown values
  known_data <- data[data$outcome != 0,]

  if (is_logistic_model == 0) {
    model <- lm(outcome ~ ., data = known_data)
  }
  else {
    # convert values
    outcome_vals <- known_data$outcome
    outcome_vals <- replace(outcome_vals, outcome_vals == 1, 0)
    outcome_vals <- replace(outcome_vals, outcome_vals == 2.05, 0)
    outcome_vals <- replace(outcome_vals, outcome_vals == 3.58, 1)
    outcome_vals <- replace(outcome_vals, outcome_vals == 4.7, 1)
    outcome_vals <- replace(outcome_vals, outcome_vals == 5, 1)

    # change outcome values to binary
    known_data$outcome <- outcome_vals
    known_data <- known_data |> mutate(outcome = as.factor(outcome))

    model <- glm(outcome ~ ., data = known_data, family = 'binomial')

    # get C score
    predictions <- predict(model, type = 'response')
    roc_score <- roc(known_data$outcome, predictions, levels = c(0, 1), direction = '<')
    c_score <- roc_score$auc * 100
    print(sprintf('C Score -> %.2f', c_score))
  }
  return(model)
}

```

Figure 7: First Logistic Regression Model Function Code

```

# function to create regression models given numeric data and the coefficient values
create_second_model_overall <- function(dataframe, model_result, is_logistic_model, desired_p_val) {
  # get significant comorbidities from first model
  model_coef_df <- as.data.frame(coef(summary(model_result)))[-1,]
  if (is_logistic_model == 1) {
    signif_comor_df <- model_coef_df[model_coef_df[, 'Pr(>|z|)'] < desired_p_val,]
  }
  else {
    signif_comor_df <- model_coef_df[model_coef_df[, 'Pr(>|t|)'] < desired_p_val,]
  }
  signif_comors <- rownames(signif_comor_df)
  num_signif_comor <- length(signif_comors)
  print(sprintf('Number of Significant Non-Zero Comorbidities from Preliminary Model -> %i', num_signif_comor))

  # check if age is included in final model
  is_age_included = 'age_years' %in% signif_comors

  # check presence of age for variable types
  if (is_age_included == FALSE) {
    # remove '1' from end of coefficient names if exists
    clean_comors <- c()
    for (val in signif_comors) {
      clean_comors <- append(clean_comors, substr(val, 1, nchar(val)-1))
    }
    # subset data
    data <- dataframe |>
      select(c(clean_comors, outcome)) |>
      mutate(across(-outcome, as.factor))
  }
  else {
    # remove '1' from end of coefficient names if exists
    clean_comors <- c('age_years')
    for (index in 2:length(signif_comors)) {
      val <- signif_comors[index]
      clean_comors <- append(clean_comors, substr(val, 1, nchar(val)-1))
    }
    # subset data
    data <- dataframe |>
      select(c(clean_comors, outcome)) |>
      mutate(across(-c(outcome, age_years), as.factor))
  }
  print(sprintf('Variables in final dataset (including outcome) -> %i', dim(data)[2]))

  # drop unknown values
  known_data <- data[data$outcome != 0,]

  if (is_logistic_model == 0) {
    model <- lm(outcome ~ ., data = known_data)
  }
  else {
    # convert values
    outcome_vals <- known_data$outcome
    outcome_vals <- replace(outcome_vals, outcome_vals == 1, 0)
    outcome_vals <- replace(outcome_vals, outcome_vals == 2.05, 0)
    outcome_vals <- replace(outcome_vals, outcome_vals == 3.58, 1)
    outcome_vals <- replace(outcome_vals, outcome_vals == 4.7, 1)
    outcome_vals <- replace(outcome_vals, outcome_vals == 5, 1)

    # change outcome values to binary
    known_data$outcome <- outcome_vals
    known_data <- known_data |> mutate(outcome = as.factor(outcome))

    model <- glm(outcome ~ ., data = known_data, family = 'binomial')

    # get C score
    predictions <- predict(model, type = 'response')
    roc_score <- roc(known_data$outcome, predictions, levels = c(0, 1), direction = '<')
    c_score <- roc_score$auc * 100
    print(sprintf('C Score -> %.2f', c_score))
  }
  return(model)
}

```

Figure 8: Second Logistic Regression Model Function Code


```

# overall function to make more usable
model_main <- function(numeric_dataframe, outcome_variable, is_binary_coef, is_logistic_model, is_lambda_min,
  desired_p_val = 0.05,
  max_df = dim(numeric_dataframe)[2]) {

  # create dataframe for rest of function
  outcome_var_list <- c('general_mental_health', 'general_physical_health',
    'general_quality', 'general_social', 'general_health')
  remove_outcome_var_list <- outcome_var_list[outcome_var_list != outcome_variable]

  # remove unwanted outcome variables
  numeric_removed <- numeric_dataframe |> select(~all_of(remove_outcome_var_list))

  # drop values with unknown outcome
  numeric_final <- numeric_removed[numeric_removed[outcome_variable] != 0,]

  # change name of outcome variable
  colnames(numeric_final)[ncol(numeric_final)] <- 'outcome'

  # run lasso regression model
  lasso_coef <- get_lasso_coef_overall(numeric_final, is_binary_coef, is_lambda_min, max_df)

  # create appropriate model wanted [first]
  first_model <- create_first_model_overall(numeric_final, lasso_coef, is_logistic_model)

  # create appropriate model wanted [second]
  second_model <- create_second_model_overall(numeric_final, first_model, is_logistic_model, desired_p_val)
  return(second_model)
}

```

Figure 9: Overall Model Function Code

```

# try with various values
test_mod <- model_main(numeric_dataframe = test,
  outcome_variable = 'general_social',
  is_binary_coef = 1,
  is_logistic_model = 1,
  is_lambda_min = 0
  , desired_p_val = 0.001
  #, max_df = 25
)

```

```

y_data
  1  2.05  3.58  4.7    5
583 2829 8355 14535 9475
y_data
  0    1
3412 32365
[1] "1 SE Lambda -> 0.005716"
[1] "Number of Non-Zero Comorbidities from Lasso Model -> 33"
[1] "C Score -> 75.85"
[1] "Number of Significant Non-Zero Comorbidities from Preliminary Model -> 20"
[1] "Variables in final dataset (including outcome) -> 21"
[1] "C Score -> 75.18"

```

Figure 10: Example Execution of Overall Model Function

While the overall model function is running, a couple of things are printed to enable some quality assurance along with some intermediate steps to ensure that everything is running smoothly. It first prints the cross tabulations of the outcome variable and then the number of non-zero comorbidity coefficients from the lasso model. After that, it prints the C-score from the first logistic regression model that uses those non-zero comorbidities. Next, it prints how many significant comorbidities exist in that first regression model, and then prints the C-score of the final regression model using those significant variables. As seen in this example, the first regression model performs marginally better in terms of C-scores, but includes 13 more covariates.

The next aspect that was looked at was the presence status of each comorbidity for each health outcome. The idea was to see if there were a good amount of comorbidities shared among

all of them. However, there were only a handful that were kept in the final model for all of the outcomes.

```
get_comor_presence <- function(numeric_dataframe) {
  outcomes <- c('general_mental_health', 'general_physical_health',
               'general_quality', 'general_social', 'general_health')

  # output dataframe
  res_df <- as.data.frame(colnames(numeric_dataframe[1:(length(numeric_dataframe)-5)]))
  colnames(res_df) <- c('Comorbidity')

  for (outcome in outcomes) {
    mod <- model_main(numeric_dataframe = numeric_dataframe,
                     outcome_variable = outcome,
                     is_binary_coef = 1,
                     is_logistic_model = 1,
                     is_lambda_min = 0,
                     desired_p_val = 0.001)

    comors <- rownames(as.data.frame(coef(summary(mod)))[-1,])
    num_comors <- length(comors)

    # check if age is included in final model
    is_age_included = 'age_years' %in% comors

    # check presence of age for variable types
    if (is_age_included == FALSE) {
      # remove '1' from end of coefficient names if exists
      clean_comors <- c()
      for (val in comors) {
        clean_comors <- append(clean_comors, substr(val, 1, nchar(val)-1))
      }
    } else {
      # remove '1' from end of coefficient names if exists
      clean_comors <- c('age_years')
      for (index in 2:length(comors)) {
        val <- comors[index]
        clean_comors <- append(clean_comors, substr(val, 1, nchar(val)-1))
      }
    }
    print(sprintf('Outcome: %s - Number of Comorbidities: %i', outcome, length(clean_comors)))
    writeLines('\n')

    # get indices of where final model comorbidities are and make value 1
    predictor_vars <- res_df$Comorbidity
    is_included <- rep(0, length(predictor_vars))
    comor_indices <- match(clean_comors, predictor_vars)
    is_included[comor_indices] <- 1

    # add 'included' to result dataframe
    res_df[outcome] <- is_included
  }
  return(res_df)
}
```

Figure 11: Comorbidity Presence by Health Outcome Function Code

From here, processing the output from the final regression model begins by making the coefficient values an integer so that a scoring system could be created.

Comorbidity	general_mental_health	general_physical_health	general_quality	general_social	general_health
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
adhd	1	0	1	1	0
acid_reflux	0	1	1	0	1
acne	0	1	0	1	1
alcohol_or_drug	0	0	0	0	0
allergies	1	0	0	0	0
anemia	0	1	0	0	1
anxiety	1	0	0	0	0
asthma	0	1	0	0	1
astigmatism	0	0	0	0	1
atrial_fibrillation	0	0	0	0	1

Figure 12: Example Output of Comorbidity Presence

```

round_coef <- function(model) {
  # get coefficients
  coef_df <- as.data.frame(coef(summary(model)))[-1,]

  # check if age is included in final model
  comors <- rownames(coef_df)
  is_age_included = 'age_years' %in% comors

  # check presence of age for variable types
  if (is_age_included == FALSE) {
    # remove '1' from end of coefficient names if exists
    clean_comors <- c()
    for (val in comors) {
      clean_comors <- append(clean_comors, substr(val, 1, nchar(val)-1))
    }
  } else {
    # remove '1' from end of coefficient names if exists
    clean_comors <- c('age_years')
    for (index in 2:length(comors)) {
      val <- comors[index]
      clean_comors <- append(clean_comors, substr(val, 1, nchar(val)-1))
    }
  }

  rownames(coef_df) <- clean_comors

  min_abs_val <- min(abs(coef_df['Estimate']))
  coef_estimates <- coef_df['Estimate']

  scaled_vals <- c()
  rounded_vals <- c()

  for (est in coef_estimates) {
    new_val <- est/min_abs_val
    rounded_val <- round_any(new_val, 1, round) * -1

    scaled_vals <- append(scaled_vals, new_val)
    rounded_vals <- append(rounded_vals, rounded_val)
  }

  coef_df['scaled_vals'] <- scaled_vals
  coef_df['rounded_vals'] <- rounded_vals
  res_df <- coef_df |> select(Estimate, scaled_vals, rounded_vals)
  return(res_df)
}

```

Figure 13: Scoring System Creation Function Code

```
rounded_res <- round_coef(test_mod)
head(rounded_res)
```

A data.frame: 6 x 3

	Estimate	scaled_vals	rounded_vals
	<dbl>	<dbl>	<dbl>
acid_reflux	-0.3400898	-2.395906	2
acne	0.3673980	2.588290	-3
anemia	-0.3719151	-2.620113	3
asthma	-0.2785996	-1.962712	2
astigmatism	0.1419462	1.000000	-1
atrial_fibrillation	-0.7012567	-4.940298	5

Figure 14: Example Output of Scoring for some Comorbidities

With all of the scoring complete, the next aspect included creating test frameworks for testing, which would compare performance of the various indices. At this point in the project, datasets to get the Charlson and Elixhauser scores have not been created, so true testing has not been done.

```

yami_index_test <- function(dataframe, points_array) {
  yami_scores <- c()

  # iterate through each row
  for (row_index in 1:dim(dataframe)[1]) {
    row_vals <- unlist(dataframe[row_index,], use.names = FALSE)
    row_yami_score <- 0

    # iterate through row values/scores
    for (col_index in 1:length(points_array)) {
      point_val <- points_array[col_index]
      variable_val <- row_vals[col_index]
      yami_point_val <- point_val * variable_val
      row_yami_score <- row_yami_score + yami_point_val
    }

    # append YAMI score
    yami_scores <- append(yami_scores, row_yami_score)
  }
  dataframe['yami_score'] <- yami_scores
  return(dataframe)
}

yami_index_test(yami_test[1:10,], yami_score_vals)

```

eddiabetes	rheumatoid_arthritis	sleep_apnea	spine_muscle_bone	type_2_diabetes	race_black	race_white	ethnicity_hispanic_latino	general_health	yami_score
<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
0	0	0	0	0	0	0	0	1	-2
0	0	0	0	0	0	0	0	0	-6
0	0	1	0	0	0	0	0	0	-9
0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	-7
0	0	0	0	0	0	0	0	1	-6
0	0	0	0	1	0	0	0	0	-6
0	0	0	0	0	0	0	0	1	-2
0	1	0	1	0	0	0	0	0	-15

Figure 15: YAMI Index Score Calculation Function and Example Scoring

To truly compare the indices, cross-validated logistic regression models are utilized for each index. The function creates a defined amount of folds and gets the corresponding C-scores for each fold for each index, printing an overall ROC curve of all folds at the end.

```

calculate_index_c_scores <- function(dataframe) {
  # create models
  charlson_model <- glm(general_health ~ charlson_score, data = dataframe, family = 'binomial')
  elixhauser_model <- glm(general_health ~ elixhauser_score, data = dataframe, family = 'binomial')
  yami_model <- glm(general_health ~ yami_score, data = dataframe, family = 'binomial')

  # get model predictions
  charlson_pred <- predict(charlson_model, type = 'response')
  elixhauser_pred <- predict(elixhauser_model, type = 'response')
  yami_pred <- predict(yami_model, type = 'response')
  actual_values <- dataframe$general_health

  # calculate C scores
  charlson_c <- roc(actual_values, charlson_pred, levels = c(0, 1), direction = '<', percent = TRUE)$auc
  elixhauser_c <- roc(actual_values, elixhauser_pred, levels = c(0, 1), direction = '<', percent = TRUE)$auc
  yami_c <- roc(actual_values, yami_pred, levels = c(0, 1), direction = '<', percent = TRUE)$auc

  charlson_rounded <- round(charlson_c, 2)
  elixhauser_rounded <- round(elixhauser_c, 2)
  yami_rounded <- round(yami_c, 2)

  print(sprintf('Charlson C Score -> %.2f', charlson_rounded))
  print(sprintf('Elixhauser C Score -> %.2f', elixhauser_rounded))
  print(sprintf('YAMI C Score -> %.2f', yami_rounded))

  return_obj <- list('charlson_c' = charlson_rounded,
                    'elixhauser_c' = elixhauser_rounded,
                    'yami_c' = yami_rounded,
                    'charlson_pred' = charlson_pred,
                    'elixhauser_pred' = elixhauser_pred,
                    'yami_pred' = yami_pred,
                    'actual_values' = actual_values)

  return(return_obj)
}

```

Figure 16: Function to Create Model for each Index and get C-Score

```

index_cv_c_scores <- function(dataframe, number_of_folds) {
  set.seed(123)
  all_charlson_c <- c()
  all_elixhauser_c <- c()
  all_yami_c <- c()

  all_charlson_pred <- c()
  all_elixhauser_pred <- c()
  all_yami_pred <- c()

  actual_values <- c()

  # split indices of data by amount of folds
  fold <- createFolds(dataframe$general_health, k = number_of_folds, list = TRUE, returnTrain = FALSE)

  # iterate through folds and calculate C score for each index
  for (fold_index in 1:number_of_folds) {
    print(sprintf('Fold %i out of %i', fold_index, number_of_folds))
    # subset data by fold indices
    fold_indices <- unlist(fold[fold_index], use.names = FALSE)
    fold_data <- dataframe[fold_indices,]

    # calculate C scores
    c_scores <- calculate_index_c_scores(fold_data)
    all_charlson_c <- append(all_charlson_c, c_scores$charlson_c)
    all_elixhauser_c <- append(all_elixhauser_c, c_scores$elixhauser_c)
    all_yami_c <- append(all_yami_c, c_scores$yami_c)

    # to account for randomized observation order from folds
    all_charlson_pred <- append(all_charlson_pred, c_scores$charlson_pred)
    all_elixhauser_pred <- append(all_elixhauser_pred, c_scores$elixhauser_pred)
    all_yami_pred <- append(all_yami_pred, c_scores$yami_pred)
    actual_values <- append(actual_values, c_scores$actual_values)

    writeLines('\n')
  }

  # get average C scores
  avg_charlson <- round(mean(all_charlson_c), 2)
  avg_elixhauser <- round(mean(all_elixhauser_c), 2)
  avg_yami <- round(mean(all_yami_c), 2)

  print(sprintf('Average Charlson C Score -> %.2f', avg_charlson))
  print(sprintf('Average Elixhauser C Score -> %.2f', avg_elixhauser))
  print(sprintf('Average YAMI C Score -> %.2f', avg_yami))

  # plot ROC curves for each index
  plot.roc(actual_values, all_charlson_pred, percent = TRUE,
    print.auc = TRUE, main = 'Charlson Index ROC Curve', levels = c(0, 1), direction = '<')
  plot.roc(actual_values, all_elixhauser_pred, percent = TRUE,
    print.auc = TRUE, main = 'Elixhauser Index ROC Curve', levels = c(0, 1), direction = '<')
  plot.roc(actual_values, all_yami_pred, percent = TRUE,
    print.auc = TRUE, main = 'YAMI Index ROC Curve', levels = c(0, 1), direction = '<')
}

```

Figure 17: Cross-Validated Modeling Function

```
index_cv_c_scores(final_index_data, 5)
```

```
[1] "Fold 1 out of 5"  
[1] "Charlson C Score   -> 66.26"  
[1] "Elixhauser C Score -> 65.97"  
[1] "YAMI C Score       -> 77.64"
```

```
[1] "Fold 2 out of 5"  
[1] "Charlson C Score   -> 68.38"  
[1] "Elixhauser C Score -> 66.96"  
[1] "YAMI C Score       -> 79.51"
```

```
[1] "Fold 3 out of 5"  
[1] "Charlson C Score   -> 66.89"  
[1] "Elixhauser C Score -> 65.72"  
[1] "YAMI C Score       -> 77.49"
```

```
[1] "Fold 4 out of 5"  
[1] "Charlson C Score   -> 66.02"  
[1] "Elixhauser C Score -> 65.30"  
[1] "YAMI C Score       -> 77.28"
```

```
[1] "Fold 5 out of 5"  
[1] "Charlson C Score   -> 68.18"  
[1] "Elixhauser C Score -> 67.38"  
[1] "YAMI C Score       -> 78.59"
```

```
[1] "Average Charlson C Score   -> 67.15"  
[1] "Average Elixhauser C Score -> 66.27"  
[1] "Average YAMI C Score       -> 78.10"
```

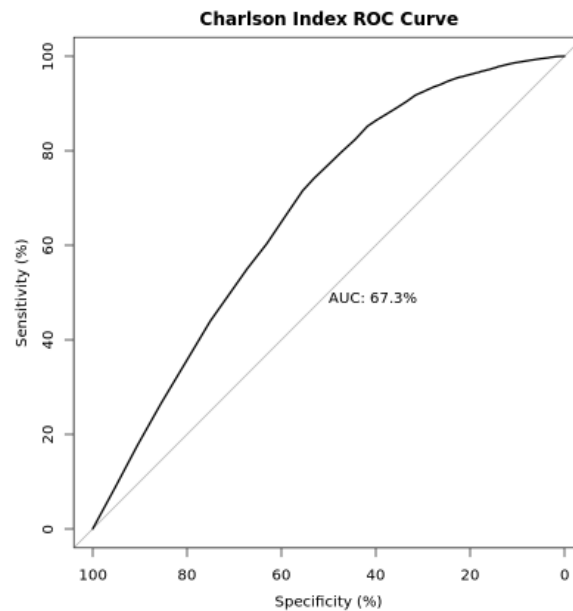


Figure 18: Example Output for Testing Framework

Discussion

Due to the nature of the All of Us data regarding building datasets and the available data elements and variables, conclusive testing regarding the possible benefits of the YAMI index has not been completed. Currently, test datasets to calculate the Charlson and Elixhauser index scores for each person have not been created and my boss is working on a solution to enable testing. However, a test framework and the corresponding functions needed have been created in anticipation of testing.

This test framework first creates a test dataset that has data containing each person's Charlson, Elixhauser, and YAMI index score along with their corresponding overall health information. With this, a function to get cross validated C scores is run. The function splits the data given a user-defined number of folds and then for each data fold, a logistic model is created for each of the indices. The model equation used each index's scores as the independent variable and the health variable as the dependent variable. Each models' fold C score would then be found and printed, along with the corresponding ROC curves (all values) at the end of the function, so that the different folds and indices' C scores can be compared.

```
[1] "Fold 5 out of 5"  
[1] "Charlson C Score    -> 68.18"  
[1] "Elixhauser C Score -> 67.38"  
[1] "YAMI C Score       -> 78.59"
```

```
[1] "Average Charlson C Score    -> 67.15"  
[1] "Average Elixhauser C Score -> 66.27"  
[1] "Average YAMI C Score       -> 78.10"
```

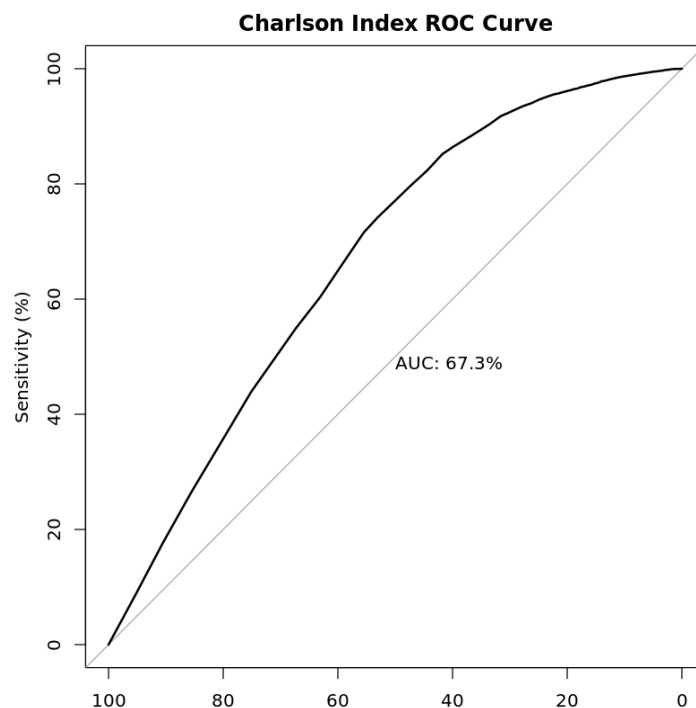


Figure 19: Example Output of Index Comparison Testing

If it is proven that the YAMI index performs better at predicting the overall health of young adults than the Charlson and Elixhauser indices, there may be numerous benefits. The first of which is that medical professionals would be able to get a better and more accurate sense of a person's health. Because of how the Charlson and Elixhauser indices are for all ages, they are not the most adequate for the younger population, who have very different comorbidity statuses and health conditions as compared to older people. Additionally, since they are based on mortality, the scores aren't measuring an appropriate quality for younger people. Another possible benefit may be that the comorbidities for certain overall health measures may be analyzed to find the most influential ones, which can lead to more awareness and precision medicine. For example, if it is found that something like asthma is greatly impacting overall health, then medical professionals may be able to raise awareness early on to try and mitigate its prevalence. Also, they may also be able to focus on finding ways to control the impact of asthma among younger people to help improve their quality of life. A last potential benefit is simply that the index would be a fresh and newly updated way to look at peoples' health. The Charlson index was published in 1987 and the Elixhauser index was published in 1998. While they are not overly outdated, there have been lots of medical advancements and changes in the world since then. Given this, a newly developed metric may be the best to be able to evaluate the health of people, particularly the young adult population.